

The Complete Replication Guide

Everything a human needs to know to rebuild this project from scratch — the process, the methodology, the client mapping, the criteria, the goal derivation, and the daily research discipline.

Document Replication guide (for humans)	As of 12 August 2026	Live app open-book-eta.vercel.app	Stack Static HTML/CSS/JS + two Python scripts
---	--------------------------------	---	---

Contents

- What this project is** — the one-paragraph pitch and the shell (two views × two tabs)
- Design principles** — the seven rules everything else follows from
- Architecture** — files, data flow, why there is no build step
- The client books** — eight engineered clients and what each one tests
- Goal derivation** — inferring Growth/Income/Preservation from the balance sheet

6. **The idea→client mapping engine** — four axes, a gate, and a goals multiplier

7. **Conviction scoring** — two rubrics, data-quality tags, and the Medium cap

8. **The daily sweep** — coverage spec, source rules, named authors, attribution

9. **The idea data contract** — every field of an Idea Feed (sweep) idea

10. **Integrity machinery** — honest dates, the tactical gate, chart/technicals sourcing

11. **Supporting engines** — expressions, the email engine, scanner, pre-trade

12. **Design system** — the J.P. Morgan Private Bank aesthetic

13. **Deployment** — GitHub → Vercel, and the one rule that matters

14. **Step-by-step replication checklist**

15. **Calibration notes and known limitations**

1 · What this project is

Open Book is a private-bank investment-desk simulator with a two-sided premise: a **Solutions desk** that manufactures and curates a daily trade-idea feed, and an **Advisor** who consumes it against a live book of eight clients. It runs as one page — `index.html` + `openbook.js` — with two header **views** (Advisor · Solutions) and two **tabs** (Idea Feed · Advisor Book), plus a light/dark theme, a "Search everything" **■K** command palette, and an embedded legacy app for the deep client pages. The point of the project is not the ideas themselves — it is the **discipline around them**: every idea is scored by a published rubric, mapped to clients by a transparent engine that reads their real holdings, dated honestly by a fingerprint ledger, and sourced under a two-source rule with citation integrity enforced by a validator.

The shell — two views × two tabs

Surface	What it does	Backed by
Idea Feed (tab)	The daily sweep as a social-style origination feed. A Market Brief card (sweep summary, expandable) sits on top; below it, a three-column layout: a left rail with the Top 3 tiles (ranked by breadth — how many clients the live engine flags — then conviction; a sweep-pinned idea leads as DESK PICK ; tap to focus the feed) and the Client Toolkit (tap a client to build their email); the middle feed with search, asset-class chips and a client filter; and a right-rail Ask your book panel (Morgan AI, answered live from the engines — "it never invents a number").	<code>today_focus.js</code> , <code>mapping.js</code> , <code>openbook.js</code> , <code>morgan.js</code> , <code>email.js</code>

Idea cards (in the feed)	Each idea renders as a post: an author identity (<i>Solutions Desk</i> for on-theme sweep ideas, <i>Claude's Views</i> for off-theme ones, <i>You — Solutions view</i> for advisor drafts), a "posted Nd ago / updated" chip driven by the honest-date ledger, the CONV ring (the desk's rubric-computed conviction /100 — click for the scoring breakdown), ticker/asset tags, headline, thesis, a RECOMMENDATION block (the tradeStatement — Solutions can tweak the implementation text advisors and emails use), an inline sourced chart, a "Via ..." attribution line (only actually-read view-sources; own calls say "<i>Own view — no outside research leaned on</i>"), ♥ save with engagement counters (liked / drafted / checked by N advisors), and two actions: Idea Suitability (the per-client fit flowchart — flagged clients with fit scores and the plain-English why; suppressed clients shown with their MiFID reason) and Email (the book-aware letter composer).	<code>openbook.js</code>, <code>charts.js</code>, <code>mapping.js</code>, <code>email.js</code>
Solutions view (header switch)	The manufacturing side. Sees the <i>whole</i> board including unapproved ideas, with a curation bar ("You curate this feed"): ■ Add idea opens a composer (name, ticker, asset class, goal bucket, direction, thesis, one-sentence recommendation, expressions) — drafts carry the advisor's name, are <i>not</i> desk-scored (a DRAFT pill instead of a CONV ring) but map to clients through the same live fit engine; each sweep card carries an ✓ Approve / ✕ Discard gate (only approved ideas reach the Advisor view; custom drafts are locked approved), an ✎ Edit implementation control, and a remove/restore flow. The Advisor view carries zero curation UI.	<code>openbook.js</code> (localStorage-persisted approval/edit state)

Advisor Book (tab)	Coverage at a glance — a grid of client tiles; tapping one embeds that client's full current-portfolio report (<code>portfolio.html?embed=1&client:=...</code>) in-shell with a "← All clients" bar: allocation vs the derived goal target, the desk agenda, mapped ideas with reasoning, holdings. The legacy app (<code>classic.html</code> , the original Brokerage Playground) survives in embed mode for the deep pages, and <code>onepager.html</code> renders a printable client one-pager.	<code>data.js</code> , <code>goals.js</code> , <code>scanner.js</code> , <code>portfolio.html</code> , <code>classic.html</code>
---------------------------	--	--

The two-axis idea philosophy. "How good is the idea" (conviction) and "how right is it for this client" (client fit) are deliberately separate scores computed by separate engines. A brilliant idea can be wrong for a book; a mediocre idea can be exactly what a gap needs. Conflating them is the cardinal sin the project is built to avoid.

2 • Design principles

1. **Derived, never hand-picked.** Idea→client links are computed live against the current books. No hand-curated "recommended for" lists that drift out of sync with reality.

2. **No black boxes.** Every score decomposes into visible, named components — the mapping axes open on click with per-axis notes; conviction pillars carry one-line reads; goal derivation lists its drivers in plain English.

3. **Facts are tagged.** Every datapoint in an idea is tagged `sourced` (≥2 independent sources) or `estimated`. A fact that can't be verified is labelled, not laundered.

4. **Citation integrity is a hard rule.** Never attribute a view to a named author whose content was not actually read during the sweep. Every view-citation carries a `basis` field stating what was read and when; the validator rejects view-citations without one. An owned original call (`ownView: true`) beats a borrowed authority every time.

5. **Honest dates.** An idea's "posted" date only moves when its *argument* changes (fingerprint ledger). A no-op re-commit never makes a stale idea look fresh.

6. **Charts are market data or they don't ship.** Any chart without a fetched, provenance-stamped series is stripped by the validator. No hand-drawn "indicative" paths.

7. **Attribution discipline.** Don't blame the scariest headline for a move — name the verified driver. Big single-name earnings (a Samsung, an IBM) are flagged as drivers where they actually moved the tape.

3 • Architecture

A **static site with no build step**: open `index.html` or serve the folder (`python -m http.server 5544`). All state is in checked-in JS/JSON files plus `localStorage` for drafted views. Two Python scripts enrich the research payload; one Python script compiles it. Nothing else runs server-side — which is what makes the whole thing deployable as a free Vercel static site and auditable file-by-file.

File	Role
<code>index.html</code> + <code>openbook.js</code> + <code>openbook.css</code>	The shell: sticky header (Advisor/Solutions view switch, light/dark theme with a pre-paint apply + iframe sync, ■K command palette, as-of stamp), the Idea Feed (brief, Top 3, toolkit, feed, Ask-your-book rail), the Solutions curation layer (approve/discard gate, idea composer, implementation edits), the Advisor Book client grid and the embed plumbing
<code>classic.html</code> + <code>app.js</code> + <code>styles.css</code>	The legacy Brokerage Playground app (the original four-tab UI), kept fully working and hosted in <i>embed mode</i> (masthead/tabs hidden, theme-synced) for the deep client pages; also openable standalone
<code>portfolio.html</code> / <code>onepager.html</code>	Per-client current-portfolio report (what an Advisor Book tile embeds) / printable client one-pager
<code>data.js</code>	Seed themes, standing theme ideas, the eight client books (positions, splits, goals, liabilities), synthetic 24-month sector history, MiFID complexity taxonomy
<code>goals.js</code>	Goal-inference engine <code>deriveGoals()</code> — the 3-bucket goal vector, willingness blend, goal-alignment multiplier
<code>mapping.js</code>	The single idea→client scorer <code>scoreIdeaForClient()</code> — four axes, tradability gate, tiers
<code>scanner.js</code>	Portfolio scan (11 severity-ranked findings rules) + thin facades over the mapping engine for the Views tab
<code>expressions.js</code>	"How to express it" knowledge base — every structure (collar, autocall, call spread, DCD...) with mechanics, example terms, pros/cons, and client-context lines
<code>email.js</code>	Book-aware client-email engine — turns any idea × client into a personalised letter with a real-holding hook, concrete implementation terms, balanced risk line and disclosure

<code>today_focus.json</code>	Raw research payload from the daily sweep (the only file the sweep writes)
<code>tv_technicals.py</code>	Enriches the payload: rewrites each idea's Technical pillar from TradingView's public scanner (MA alignment, Bollinger, RSI) and flips its dq to sourced; adds RSI colour to Positioning without flipping its dq
<code>fetch_chart_series.py</code>	Fills each chart block's series with real Yahoo Finance daily closes + provenance stamp
<code>build_today_focus.py</code>	Deterministic compiler + validator: rubric scoring, coverage checks, tactical gate, date ledger, output <code>today_focus.js</code>
<code>today_focus.js</code>	<code>window.TODAY_FOCUS</code> — what the app loads. Auto-generated; never hand-edited
<code>today_focus.ledger.json</code>	Committed fingerprint ledger keeping postedAt/updatedAt honest
<code>methodology.js</code>	<code>window.METHODOLOGY</code> — reads every constant LIVE from the engines so the in-app methodology can't drift from the code
<code>morgan.js</code> / <code>charts.js</code>	The "Ask your book" Morgan AI panel (answers from METHODOLOGY + the live engines) and the inline spark/band chart renderer

The daily data flow

1. SWEEP (Claude Code, on demand): read the tape + named authors → write `today_focus.json` (ideas, facts tagged, sources, pillar first-pass) 2. `python tv_technicals.py` # Technical pillar ← TradingView (sourced) 3. `python fetch_chart_series.py` # `chart.series` ← Yahoo daily closes (sourced) 4. `python build_today_focus.py` # validate + score + gate + date ledger → `today_focus.js` 5. `git commit` + `git push` # Vercel redeploys; client mapping stays live in-browser

The client mapping is intentionally **not** baked into the generated file — it is computed in the browser against the current Advisor Book, so flagged clients can never drift from the real books.

4 • The client books

Eight deliberately different clients, each engineered so the engines demonstrably do something different. Aurora is the "real" anchor book; the others are consistent, distinct private-bank books. Positions are canonical: each book's weights sum to 100 and the asset-class **split** equals the per-class sum of positions, so every engine reads one reconciled truth.

Client	Type / AUM	Engineered to test
Aurora	Retail · €50.2m	The anchor book: a 25.8% Micron position on a ~4x gain, 72% USD vs an EUR base, an underwater bond — concentration protection, FX mismatch, tax-loss harvesting, and the Retail/OTC suppression rule (a collar suggestion must be suppressed with a reason, not silently dropped)
Fable	Professional · \$38m	Options-fluent concentrated AI book (NVDA/AVGO/MU/ASML/MSFT) — monetisation ideas: collars, overwrites, autocalls on names already held
Scott	Retail · \$124m	The big conservative income estate — bond swaps, tax-loss harvesting (PFE, O), cash laddering; income ideas should fit, growth ideas shouldn't
Amar	Professional · \$71m	Real-assets family office with a 16% Bitcoin position down 35% — the rehabilitation case (harvest + structured re-entry); gold ideas should score 0 gap-fit because his Preservation bucket is already full
Jacob	Professional · \$96m	Liability-aware planning: \$19m floating mortgage + \$2.4m Q3 tax bill vs 5% cash — the goal engine must push Income/Preservation despite a balanced book
Prahnav	Retail · \$44m	Goals-aligned top-down but concentrated bottom-up (NVDA +279%, CEG +181%) — the Retail-collar problem: the right trade needs reclassification or a packaged-note alternative

Mitch ("morgan")	Professional · \$38m	Max-aggressive founder, ~92% risk assets, no funding goal on file — the goal engine must infer aggressive growth from stated risk + the book (source "stated-risk"), not quietly de-risk him
Fotis ("tejpaul")	Retail · \$60m	Never risk-profiled, no goals, no liabilities — the pure "revealed" case: the balanced book alone must yield a moderate goal vector (source "revealed", lowest confidence)

Client schema (what you must author per client)

- `id`, `name`, `ccy`, `aum`, `classification` (Retail | Professional), `mifid`, `relationship`, `risk` (free text — parsed for stated willingness), `profile`, `summary`

- `split` — asset-class percentages (must reconcile with positions)

- `goals` — objective, horizon ("Long-term · 7–10 yrs" — parsed), optional funding {headline ("Grow to €70m by 2035" / "Fund \$3.6m/yr..." — parsed for required return / income need), current, target, unit, status}

- `liabilities` (optional) — amount + free-text note; "tax/due/Q3" reads as a near-term bill → Preservation; "mortgage/loan/credit" reads as serviceable debt → Income

- `positions` — name, ticker, assetClass, sector, ccy, weightPct, pnlPct, entryDate, entrySpot, maturity, note. Alternatives may carry `altKind`: "uncorrelated" | "directional" (hedge funds → Preservation ballast; crypto/VC → Growth)

- `sectorHistory` — synthesised deterministically from the positions (24 monthly points per sector; winners ramp, losers fade; index 0 equals the current allocation exactly). Swap for real history when available; nothing else changes

5 • Goal derivation (`goals.js`)

Real clients never hand you a target allocation — they say "I want to grow capital", "I have a \$19m mortgage", "I need \$0.8m/yr". `deriveGoals(client)` infers a three-bucket goal vector — **Growth • Income • Preservation** — from both sides of the balance sheet plus revealed risk appetite, in the spirit of Chhabra's risk-allocation framework: *fund the floor first, then deploy the surplus by how hard the money must work*.

Inputs (all measured off the client object)

- **Horizon** — parsed from the free-text horizon ("Drawdown • 0–5 yrs" → phase=drawdown, H=2.5y; "Long-term • 7–10 yrs" → accumulation, H=8.5y).
- **Funding goal** — parsed from the headline: a value target ("grow to \$75m by 2034") becomes a **required return** = $(\text{target}/\text{current})^{(1/\text{years})} - 1$; an income target ("fund \$3.6m/yr") becomes an **income need** as % of AUM; both can coexist.
- **Liabilities** — near-term bills (tax, settlements) as % of AUM; debt load; annual debt service (rate defaults to 6%).
- **Willingness** — blended 50/50 from **stated** (parsed from the risk string: aggressive 0.95, growth 0.80, moderate 0.50, conservative 0.30) and **revealed** (read off the book: $0.5 \times \text{risk-asset share} + 0.3 \times \text{low-defensiveness} + 0.2 \times \text{single-name concentration}$ appetite). If nothing is stated, revealed stands alone.

The scoring (each bucket = sum of named, inspectable terms)

Preservation = 8 (base) + 1.0·nearBill% + 0.3·debtLoad% + 1.0·max(0, topName% - 15) + 15 (if drawdown phase) + 3·max(0, 5 - H) + 28·(1 - willingness) Income = 8 (base) + 8·incomeNeed% + 8·debtService% + 0.6·debtLoad% + 15 (if drawdown phase) + 18·(1 - willingness) Growth = 8 (base) + 5·requiredReturn% + 3·max(0, H - 6) + willingness × (30 if a required-return signal exists, else 45) vector = largest-remainder normalisation of the three raw scores to exactly 100

Three subtleties that make it work. (1) **Anti-circularity:** the book sets the willingness *scalar*, never the target vector — the target is still built by the bucket logic, so the gap between "have" and "need" stays meaningful. (2) **Bidirectional willingness:** a low appetite must *add ballast* (the caution terms), not merely shrink Growth — otherwise a cautious client with no liabilities comes out growth-heavy by default. (3) **Confidence reweight:** with no goal signal, the willingness coefficient rises from 30 to 45 — a missing written goal must never quietly de-risk an obviously aggressive book.

The result carries a **source** ("stated-goal" 0.9 confidence · "stated-risk" 0.7 · "revealed" 0.55), the plain-English driver list surfaced in the UI ("Needs ~7.1%/yr over 8y → Growth", "\$19m debt load → Income/Preservation matching"), and every component number. A manual `goalOverride` pins a vector and short-circuits the model. The **current** book is folded into the same three buckets for the "now vs goal" bar: equity + directional alternatives → Growth; fixed income, credit, real assets → Income; cash, gold, uncorrelated alternatives → Preservation; structured notes by purpose (keyword-classified: buffer/protect → Preservation, autocall/coupon → Income, else Growth).

6 · The idea→client mapping engine (`mapping.js`)

`scoreIdeaForClient(idea, client)` is the **single scorer** — the Idea Feed's flagged clients, the Top 3 breadth ranking, the suitability flowchart, advisor-draft mapping and the legacy pages all funnel through it, so no two tabs can ever disagree. It reads sector-level signals off the reconciled book: sector exposure, the 24-month sector history, in-sector holdings, and the parsed mandate.

The four axes (each 0–100, flat weights summing to 1.00)

Axis	Weight	Computation
Affinity fit (holdings)	0.294	$\max(0, \text{ThematicAffinity} - \text{ConcentrationPenalty})$. Affinity = recency-weighted ($\lambda=0.94$) percentile of the current sector allocation within the client's 24-month sector history — "is this a market they demonstrably live in?". Penalty = pp over the sector-comfort peg $\times 10$ (capped 100). Comfort pegs by the <i>asset's</i> goal bucket: Growth 25 · Income 15 · Preservation 12 (% of book), with optional per-sector overrides.
Mandate & Risk	0.294	$0.6 \times \text{RiskSuitability} + 0.4 \times \text{IntentFit}$. RiskSuitability = deterministic matrix of the idea's riskProfile {vol, beta, structured} vs the client's mandate class (growth mandates reward high-beta; income mandates reward low-vol; preservation punishes high-beta). IntentFit = matrix mandate $\times$ goalType (growth $\leftrightarrow$ appreciation 90, income $\leftrightarrow$ yield 90, preservation $\leftrightarrow$ protection 92, off-goal pairings lower).
Gap fit	0.235	Bucket-aware headroom: $\max(0, (\text{bucketTarget} - \text{bucketCurrent}) / \text{bucketTarget} \times 100)$ where bucketTarget is the client's <i>derived goal</i> for the idea's bucket and bucketCurrent the book's current allocation to it. Zero once the goal bucket is full — a gold idea scores 0 for a client whose Preservation is already at target. Falls back to sector headroom vs the comfort peg only when no goal target exists.
Concentration within sector	0.176	$\text{raw} = (1 - \text{HHI}) \times 100$ over the book's holdings inside the idea's sector (one name $\rightarrow 0$; five equal names $\rightarrow 80$). Inverted for fit by default: a concentrated sector needs a new name more, so $\text{fitContribution} = 100 - \text{raw}$. No in-sector holdings $\rightarrow$ neutral 50.

Combine — gate × bracket × goals multiplier

```
bracketFit = Σ (axisScore × weight) # 0-100 alignedFit = bracketFit × goalAlignment.mult #  
0.85-1.15 (goals.js) fit = tradability × alignedFit # binary MiFID gate tiers: ≥66 Strong ·  
≥48 Good · else Marginal applies: tradable AND alignedFit ≥ 45 · flagged: fit ≥ 50, top 6
```

- **The goal-alignment multiplier** (goals.js): an idea that fills a bucket where the client is *under* their derived goal is boosted (up to ×1.15); one piling into an over-goal bucket is dampened (down to ×0.85). mult = clamp(1 + gap/200).

- **The tradability gate** (binary, computed before everything): Retail client + an OTC natural expression (collar, forward, OTC option — keyword taxonomy in data.js) → fit 0, suppressed. The UI **surfaces** suppressed clients with the MiFID reason rather than silently dropping them. Structured *notes* (autocalls, buffered notes) are packaged securities Retail *can* hold — the complexity check runs structured-keywords before OTC-keywords so "HALO basket" resolves as a note, not a basket.

What deliberately moved out of client fit. House-view fit is an idea-level property, not a per-client signal — it double-counted holdings already captured by Affinity and Gap — so it lives in the conviction rubric now. Tradability used to be an axis term (a non-tradable idea could still score ~75); as a global gate it zeroes the whole fit instead.

7 · Conviction scoring — two rubrics by idea kind

Computed by `build_today_focus.py`, never hand-entered. Pillar scores and data-quality tags are authored in the research payload; the generator stamps labels and maxima, sums, scales to /100 and bands. Each pillar's core input carries `dq ∈ {sourced, estimated, unverified}`.

Earnings ideas — the print rubric (4 pillars × 0–2, /8)

Pillar	Scores 2 / 1 / 0
Asymmetry signal	Implied straddle move ÷ average absolute realised move over the trailing 4 prints: materially below 1 (market underpricing the event) → 2 · near fair → 1 · implied rich → 0. Wide estimate dispersion can also carry it.
Sell-side consensus	Majority buy + upside to mean PT + positive 30-day revisions: all three → 2 · partial → 1 · none/contradicts → 0.
Catalyst clarity	Pre-print: a dated, unpriced catalyst. Post-print: a reaction disproportionate to print quality. Clear → 2 · soft → 1 · none → 0.
Positioning & sentiment	Short interest >10% of float OR an extreme setup → 2 · notable lean → 1 · neutral/unavailable → 0.

Banding: **High** = raw ≥6, no zero pillar, all inputs sourced · **High — data gap** = same but ≥1 estimated input · **Medium** = raw 4–5, or any zero pillar, or consensus unverified · **Low** = raw <4 or two-plus zeros (excluded from the board).

Ex-earnings ideas — the seven-pillar model (/15)

Pillar (max)	What it scores
--------------	----------------

Gap / asymmetry (3)	Thesis expected move ÷ the name's normal move over the horizon: $\geq 2.0 \rightarrow 3 \cdot 1.5 - 2.0 \rightarrow 2 \cdot 1.0 - 1.5 \rightarrow 1 \cdot < 1.0 \rightarrow 0$. The target is a soft input $\rightarrow$ tagged estimated.
Catalyst (2)	A dated, hard reason it moves soon $\rightarrow 2 \cdot$ soft/undated $\rightarrow 1 \cdot$ none $\rightarrow 0$.
Consensus + confirmation (2)	Either sell-side aligned OR a defensible variant view, AND an independent signal (breakout / flows / skew). Both legs $\rightarrow 2 \cdot$ one $\rightarrow 1 \cdot$ neither $\rightarrow 0$.
Fundamental thesis (2)	Two 0/1 checks: driver specificity (named, quantified drivers vs narrative) and variant view (an articulated, computable gap vs consensus, authored in the idea's variant {street, us, gap}). Pure consensus-hugging scores 0. dq is sourced only when drivers trace to cited facts.
House-view fit (2)	Core desk theme, direct expression $\rightarrow 2 \cdot$ on-theme/adjacent $\rightarrow 1 \cdot$ off-theme or against $\rightarrow 0$.
Positioning (2)	Confirmation/veto, not a driver: a hard read (SI, CoT, flows) + a sentiment read (RSI at a crowded extreme). Crowd offside our trade $\rightarrow 2 \cdot$ neutral/mixed $\rightarrow 1 \cdot$ crowd with us (unwind risk) $\rightarrow 0$.
Technical (2)	Confirmation/veto: 50/100/200-day MA alignment, Bollinger entry, Fibonacci vs the thesis. Agrees $\rightarrow 2 \cdot$ mixed $\rightarrow 1 \cdot$ contradicts $\rightarrow 0$.

score = raw ÷ 15 × 100 • **High** ≥75 • **Medium** ≥55 • **Watch** ≥0. The split is deliberately ~70/30: thesis-bearing pillars carry 11/15, positioning + technicals 4/15 — *the view drives; technicals confirm or veto but cannot manufacture conviction*.

The data-quality cap. If any pillar's core input is estimated or unverified, the ex-earnings label cannot exceed Medium (earnings shows "High — data gap"). This is the honest-data flywheel: wiring in a sourced feed (as tv_technicals.py does for the Technical pillar) visibly unlocks higher labels. Every idea also carries a **changeMyMind** field — the condition that would invalidate the view — surfaced on the tile but deliberately not scored (a kill-switch pillar would bias the model toward dated catalyst trades over strategic holds).

8 • The daily sweep — process and rules

The sweep is run on demand by a Claude Code session (no scheduler is wired up; the cadence is a convention). It is a **whole-market** sweep, not a US-equity screen:

Coverage spec (validator-enforced warnings)

- Asset classes: equities, rates, credit, FX, commodities (+ structured where it fits); regions: US, Europe, UK, Asia.
- **FX and rates must be represented every run** — the two most-often-missed classes.
- The earnings book must carry **both stances**: pre-position (into a future print) and post-print (playing an already-reported reaction with a verified, sourced result).
- At least one **tactical** idea — a short-dated option trade with a concrete construction (strikes/tenor/premium) and a defined entry/target/stop in a `levels` block — primarily in FX.
- **Dislocation scan**: check large-cap drawdowns — a >~15% pullback in a widely-held name is a candidate idea, not noise.

Source rules

- **≥2 independent sources** for any stated fact; single-sourced figures are tagged estimated. Forward events stay future-tense.

- **Named authors, two tiers.** Core (read every sweep): Citrini Research (thematic), Serenity, TSCS, Brent Donnelly (FX), Cem Karsan (vol/dealer positioning), Joseph Wang / Fed Guy (rates & Fed plumbing), SemiAnalysis (AI capex/semis). Conditional (read when an idea touches their domain — the natural second source): The Transcript (earnings), Benn Eifert / Moontower / Tier1 Alpha (options & vol), Fabricated Knowledge (semis), Rory Johnston / Doomberg (commodities), HighYield Harry (credit), The Market Ear / Brad Setser (flows), Andy Constan / Joseph Politano (rates & macro data).

- A view-citation (kind "view") requires a **basis**: what was read, when, and what was taken from it. Fresh commentary that merely restates an unchanged thesis is not a reason to cite or re-date an idea.

- Nothing bypasses the rubric or the mapping: an author's read informs conviction; it never auto-promotes an idea.

What a sweep actually writes

The sweep overwrites `today_focus.json`: an `asOf` date, a `sweep` block (source list, the rule line, a three-part brief — summary, watch bullets, full narrative paragraphs — and a desk `note` that archives what changed on the board and why, including what was *held back* and the attribution discipline line), an `earnings` array and an `exEarnings` array of idea objects (§9), and a `meta` block stamping the technicals feed as-of. Ideas that survive from prior sweeps keep their `id` — the ledger then keeps their posted date honest; retired ideas simply drop out of the file (the ledger forgets them).

9 • The idea data contract

Every idea in `today_focus.json` is one JSON object. The load-bearing fields:

Field	Contract
<code>id</code>	Stable slug — identity for the date ledger. Keep it across sweeps while the idea lives.
<code>direction</code>	long short neutral (drives the direction-aware technical read).
<code>tradeStatement</code>	Required. One precise sentence: instrument + direction + the actual view. FX must name long-vs-short legs and spot-vs-carry.
<code>intent / action / preferredExpression / structures[]</code>	add·protect·trim·income; the action line; the lead expression; the clickable expression list (use vocabulary expressions.js resolves — "Call spread", "Zero-cost collar", "Phoenix autocall", "Buffered note", "Risk reversal", "Strangle", "Dual-currency deposit"...).
<code>name / ticker / sector / assetClass / bucket / goalType / riskProfile</code>	Classification the engines read. bucket ∈ Growth·Income·Preservation·Structured; sector "FX" and "Rates" satisfy the coverage check; riskProfile {vol, beta, structured} feeds Mandate & Risk.
<code>headline / thesis / variant {street, us, gap} / changeMyMind</code>	The argument. variant is what the thesis pillar's variant-view check reads; changeMyMind is the surfaced kill condition.
<code>themeId / offThemewhy / ownView</code>	Link to a standing desk theme (drives the card's author identity: on-theme = Solutions Desk, off-theme = Claude's Views), or an explicit off-theme reason. ownView marks a desk-original call.
<code>conviction.pillars[]</code>	Authored scores + dq + one-line notes per rubric pillar (4 for earnings, 7 for ex-earnings). Totals/labels are computed, never authored.
<code>earnings {...}</code>	Earnings ideas: reportDate, reportWhen, stance-dependent fields — pre: impliedMovePct vs historicalAvgMovePct (+ sources); post: result + resultSource="sourced" (validator-enforced) + reaction + watch.
<code>facts[] / sources[]</code>	Facts each tagged sourced estimated. Sources typed kind ∈ data·news·calendar·analysis·view; view requires basis.

tactical / trigger / triggered / levels {tenor, entry, target, stop}	The tactical contract (§10). Only triggered tacticals ship.
chart {kind, symbol, range, band, refs, caption}	Optional; series filled by fetch_chart_series.py, stripped if unsourced.
macro {metric, detail, source, watch}	Ex-earnings context block shown on the card.

10 • Integrity machinery

Honest dates — the fingerprint ledger

The build fingerprints each idea's *argument* (tradeStatement, headline, thesis, changeMyMind, direction, intent, structures, fact texts, pillar scores/notes, levels, earnings result — explicitly excluding chart series, indicator timestamps and computed totals). The committed ledger maps id → {fp, postedAt, updatedAt}: unseen id → both dates = asOf; unchanged fingerprint → both dates carry; changed fingerprint → postedAt carries, updatedAt bumps (a "· updated Nd ago" chip). The build **warns** when an idea's thesis has gone >10 days unchanged — stale commentary is surfaced, not silently aged. Price/timestamp refreshes alone do not count as change.

The tactical trigger gate

Any idea carrying a `levels` block must declare a `trigger` (the stated condition that justifies it *now*) and a boolean `triggered`. Only triggered tacticals are written to the output; untriggered ones are dropped and reported ("held back") — so the board never carries a tactical trade without a live justification, and the desk note records what the gate held back (e.g. an EUR/USD call spread waiting for its 1.17 trigger). Tactical options are Growth/appreciation by classification — a directional FX option is not a hedge.

Sourced technicals and charts

`tv_technicals.py` rewrites the Technical pillar from TradingView's public scanner (SMA50/100/200, Bollinger, RSI, Recommend.MA) direction-aware — a rising trend confirms a long and vetoes a short; neutral/range ideas are scored on tape-calmness instead. It flips only that pillar's dq to sourced; Positioning keeps its authored dq because short interest / CoT aren't in the feed — flipping it wholesale would be dishonest. `fetch_chart_series.py` fills chart series from Yahoo daily closes with a provenance stamp; the validator strips any chart without one. When a sweep runs without re-running these scripts, the output states the stale as-of honestly rather than pretending freshness.

11 · Supporting engines

- **expressions.js** — the "how to express it" knowledge base. Every structure carries: what it is, mechanics, example terms with realistic numbers, pros/cons, a complexity flag (drives the MiFID gate), goal-role scores {g, i, p}, and a context function that personalises the line to the client's actual holding. Structure names are keyword-resolved, so idea structures[] entries must use its vocabulary.

- **email.js** — turns any idea × client into a personalised letter: a real-holding hook, a tax-swap / loss-harvest / cash-redeploy / FX-sizing action, concrete implementation terms, a balanced risk line, and a disclosure. Shared by both render paths so copy can't drift.

- **scanner.js** — 11 hand-coded, severity-ranked portfolio findings (concentration, underwater tax-loss candidates, idle cash, currency mismatch, missing protection...) powering the Advisor Book agenda and next-best-action; plus thin facades over the mapping engine.

- **morgan.js** — the in-app AI assistant; answers methodology questions from `window.METHODOLOGY`, which reads constants live from the engines so it cannot drift.

- **charts.js** — renders the spark/band chart blocks inline on idea cards.

12 · Design system

Two coordinated systems, both theme-aware:

- **The shell (openbook.css):** light ground #FBFAF7 / white surfaces with a full **dark theme** ([data-theme="dark"], gold-accented) toggled in the header, persisted in `localStorage` (`ob-theme`), applied pre-paint to avoid flash and synced into the embedded iframe. Ink #17150F, charcoal #1E1B16, brown #54301A, bronze #996F3D / light bronze #C7A97A, gold #EBC98D, greens #2FBF71 / #2C7A4B (positive/fit), brick #9C4A3A, hairlines #E5E1DA/#F0ECE4, creams #F4F1EA/#EEEEAE2. Type: **Source Serif 4** (display/serif) + **Archivo** (UI sans), Google-Fonts loaded. Texture: letter-spaced uppercase kickers, colour strip-bars on cards, conviction rings, soft card shadows — an editorial-feed feel.

- **The legacy pages (styles.css — classic/portfolio/onepager):** the original printed-research-note aesthetic — espresso #29211A on warm paper, bronze #9A7B4F, hairline rules, tag chips — with its own dark-theme mapping so the embed matches the shell's theme.

- **Goal-bucket colours (shared):** Growth dark · Income bronze · Preservation green.

13 · Deployment

1. GitHub repo (private is fine) → Vercel project via the native Git integration; every push to `main` auto-deploys the static folder (no build command, output = root).

2. **The one rule:** committing locally is invisible — **always push**. The live site only changes on push.

3. Commit style: `chore: market sweep refresh YYYY-MM-DD` (one-line desk summary of what changed) — the git log doubles as the desk's sweep journal.

14 · Step-by-step replication checklist

1. **Scaffold the static app:** the shell (index.html + openbook.js + openbook.css: header with view switch/theme/■K, Idea Feed, Advisor Book grid) plus the legacy pages (classic.html + app.js + styles.css, portfolio.html, onepager.html) in embed mode.

2. **Author data.js:** themes → ideas (typed, bucketed, with structures) → the eight client books (reconciled positions) → the synthetic sector-history generator → the MiFID complexity taxonomy (structured-keywords before OTC-keywords).

3. **Build goals.js:** the parsers (horizon, funding, liabilities), the willingness blend, the three component sums, largest-remainder normalisation, drivers list, currentBuckets folding, goalAlignment multiplier. Sanity-check against the two no-goal clients: Mitch must come out aggressive-growth, Fotis moderate.

4. **Build mapping.js:** the four axes exactly as §6, the flat weights, the tradability gate, the goal-alignment multiply, tiers and gates. Verify the canonical worked examples: gold × Amar → Gap fit 0; a collar × Aurora (Retail) → suppressed with reason; five-name sector → concentration raw 80, inverted contribution 20.

5. **Build the pipeline:** build_today_focus.py (rubrics as §7 constants, validators as §8–10, ledger, tactical gate, coverage checks), tv_technicals.py, fetch_chart_series.py.

6. **Author the first today_focus.json** by running a real sweep (§8), then the three scripts; fix every validator warning rather than suppressing it.

7. **Wire the UI:** app.js render paths, drawers (per-axis breakdown, pillar tiles, expression accordion), the "How scoring works" modal reading the rubric from TODAY_FOCUS, methodology.js reading PARAMS live, email.js, morgan.js.

8. **Deploy:** GitHub → Vercel; push; verify the live URL renders the board and the mapping runs in-browser.

9. **Operate:** re-run the sweep on demand; let the ledger and validators keep you honest; retire ideas by deleting them from the payload; never edit today_focus.js by hand.

15 · Calibration notes and known limitations

- Every coefficient (goal weights, comfort pegs, axis weights, tier cut-offs) is a **reasoned calibration choice, not a fit to outcomes** — they live in PARAMS objects precisely so they are visible and tunable. The in-browser harness (score ideas against window.SEED) is how they were sanity-checked.

- AUM, currency-hedging depth, liquidity and liability-funding don't yet enter the fit score (scanner.js handles some of this separately).

- The straddle / short-interest feeds aren't wired, so those pillar inputs stay estimated — which, by design, caps most ideas at Medium until sourced feeds are added. This is a feature: the cap is the visible cost of unsourced data.

- The sector history is synthetic seed data shaped from position P&L; swap in real monthly history and nothing else changes.

- The concentration axis's invert-for-fit direction is a product decision on a single flippable flag.

Open Book · Complete Replication Guide · generated 12 August 2026 · This document describes a research/portfolio simulation project. Nothing in it is investment advice.